

category: ui-patterns

tags: [localization, i18n, avalonia, source-generation, resx, magic-string]

severity: medium

created: 2026-07-16

updated: 2026-07-16

project-origin: parking-v8-app-avalonia

Localization bằng magic string key ("Get") dễ lỗi — port sang property mạnh kiểu như bản WinForms

Tình huống gặp phải

parking-v8-app-avalonia (Avalonia) so với parking-v8-app (WinForms gốc) — cả hai đều dịch đa ngôn ngữ cho ~950 chuỗi UI.

Avalonia dùng `AppLocalizationService.Get("FrmLogin")` — key là string literal rời rạc, tra trong `Dictionary<string, string>` load từ `dictionary/*.json`. Bản WinForms KHÔNG dùng resx mà dùng class `AppResources : UIBuiltInResources` với **1 property C# thật cho mỗi key** (`KZUIStyles.CurrentResources.FrmLogin`), JSON deserialize thẳng vào property đó.

Triệu chứng / Lỗi

Gõ sai key (`"FrmLogin"`) hoặc JSON đổi tên key → `Get()` không lỗi, không cảnh báo, chỉ âm thầm fallback về key gốc hoặc bản dịch mặc định (`fallback ?? key`). Không có IntelliSense, không "Find All References", refactor-rename không hoạt động. Phát hiện được nhiều nhất là lúc QA nhìn UI thấy chữ tiếng Anh/key thô thay vì bản dịch đúng.

Khi khảo sát còn phát hiện 1 key JSON bị lỗi thật trong sản phẩm: `"FrmSelectLane: "` (dư dấu `:` và space) — do đây chỉ là string key tự do nên lỗi này lọt qua nhiều lần build/deploy mà không ai biết; code phải viết `.Replace(":", "").Trim()` để vá tạm ở call site (`SelectLaneModeWindowViewModel.cs`).

Nguyên nhân gốc rễ (Root Cause)

Thiết kế `Dictionary<string, string> Values` bên trong `AppBuiltInResources` khiến key chỉ tồn tại dưới dạng string tại runtime — không có bất kỳ ràng buộc nào giữa "key khai báo trong JSON" và "key dùng ở call site" được compiler kiểm tra.

Giải pháp

Đổi `AppBuiltInResources` từ `Dictionary<string, string> Values` sang **1 property string thật cho mỗi key trong `vi.json`** (script Python đọc `vi.json`, sinh property `public string {Key} { get; set; } = "{giá trị vi mặc định}";`). JSON parse ra `Dictionary<string, string>` rồi set vào property qua reflection cache (`ApplyValues`) — không dùng `JsonSerializer.Deserialize<AppBuiltInResources>` trực tiếp vì `CultureInfo` là `required init` không có trong JSON.

Call site: `AppLocalizationService.Get("FrmLogin")` →

`AppLocalizationService.CurrentResources.FrmLogin` (compile error nếu gõ sai/xóa property).

```
# Sinh property từ vi.json (xem script đầy đủ trong lịch sử conversation)
for key, value in vi.items():
    print(f'    public string {key} {{ get; set; }} = "{escape(value)}";')
```

1. Sửa key JSON lỗi (`"FrmSelectLane: "` → `"FrmSelectLane"`) trong TẤT CẢ file `dictionary/*.json` trước khi sinh class — script generator chỉ nhận key hợp lệ định danh C#.
2. Sinh class property từ `vi.json` (nguồn canonical vì `DefaultCulture = vi-VN`).
3. Đổi `LoadLanguagesFromJson` sang `deserialize Dictionary<string, string>` rồi gọi `resource.ApplyValues(flat, unknownKeys)` — log các `unknownKeys` không khớp property nào (bắt lỗi JSON-drift sớm thay vì im lặng).
4. Migrate call site bằng regex tự động: `AppLocalizationService.Get\("([A-Za-z_]\w*)"\)` → `AppLocalizationService.CurrentResources.\1` — CHỈ áp dụng khi arg là string literal đơn (220/~370 call site trong project này khớp mẫu này).
5. KHÔNG migrate các call site dùng key động (biến, string nội suy, hoặc key đọc từ XAML attached property như `ReportLocalization.Key`) — các chỗ này giữ nguyên `Get(string key)`, giờ được backing bởi reflection trên property thật (an toàn hơn dictionary tự do) thay vì xóa hẳn API động.

Áp dụng lại (How to reuse)

- Khi thấy pattern `XxxService.Get("LiteralKey")` lặp lại hàng trăm lần với key rời rạc trong dự án Avalonia/WinForms C# → đề xuất ngay property mạnh kiểu thay vì thêm lớp bọc enum/const (enum vẫn phải tra runtime, không có ưu điểm bằng property thật).
- Trước khi sinh code hàng loạt từ JSON/resx: LUÔN kiểm tra key có phải định danh C# hợp lệ không (`^[A-Za-z_][A-Za-z0-9_]*$`) — key lỗi kiểu `"FrmSelectLane: "` là dấu hiệu bug có thật trong dữ liệu, sửa luôn tại nguồn (JSON) trước khi generate.
- Phân loại call site TRƯỚC khi tự động thay thế bằng regex: literal-only vs dynamic-key (biến, XAML attached property, string.Format template key) — chỉ migrate literal, giữ dynamic qua API cũ.

Chú ý / Cạm bẫy (Gotchas)

- ⚠ `JsonSerializer.Deserialize<T>` KHÔNG populate được object có `required init` property không tồn tại trong JSON (ví dụ `CultureInfo`) — phải tạo instance trước rồi set property qua reflection, không deserialize thẳng vào type đích.
- ⚠ Một số JSON key không phải PascalCase chuẩn (`colNote`, `colAccessKeyName`) — giữ nguyên đúng tên đó làm property name (không tự ý đổi casing) để mapping 1:1 với JSON, tránh phải sửa lại tất cả file JSON song song.
- ⚠ Luôn build lại (`dotnet build`) ngay sau khi chạy script generator + regex migrate — vài chục warning CS8602/CS0618 có sẵn trong project là bình thường, chỉ cần `0 Error(s)`.

Tham chiếu

- Windows reference: `iParkingv8.Utility/dictionary/AppResources.cs`, `UIBuiltInResources.cs`, `KZUIStyles.cs` (project `parking-v8-app`)

- Avalonia: `src/ParkingV8.App/Localization/AppBuiltInResources.cs`,
`AppLocalizationService.cs`
- Project liên quan: parking-v8-app-avalonia